

Intern — Flow of Presentation (IMRaD, 2 hours)

Built 2026-10-06 from docs/presentation_flow.md · results measured 2026-10-06

I Introduction **M** Methods **R** Results **D** Discussion

Your workflow, cloned. A thesis defense plan for a 2-hour slot, organized as **Introduction** → **Methods** → **Results** → **Discussion**.

Where the numbers come from. Every number in the Results section comes from `scripts/thesis_figures/objective_metrics.py`, as run on 2026-10-06. **Run it again the day before the defense** and update the slides. Don't type any number into a slide by hand. Practice values still in `Final Document.docx` Chapter 4 are **not** measured. Never present them as results.

Time budget

3 min	I · Introduction 20 min	M · Methods 35 min	Bre... 5 min	R · Results 27 min	D · Discussion 15 min	Q&A 15 min
Block			Minutes	Slides	Running clock	
0. Opening			3	1–2	0:00 – 0:03	
I — Introduction			20	3–11	0:03 – 0:23	
M — Methods			35	12–27	0:23 – 0:58	
Break / buffer			5	—	0:58 – 1:03	
R — Results			27	28–41	1:03 – 1:30	
D — Discussion & Conclusion			15	42–49	1:30 – 1:45	
Open Q&A			15	50 + backup	1:45 – 2:00	

About 50 main slides plus backup slides, at roughly 2 minutes per slide. If you fall behind, take time from the Methods deep-dives (slides 21–24), not from Results.

0. Opening (3 min)

- Title slide.** Intern: a personalized GUI agent that learns a task from a user's own demonstrations. Team, adviser, date.
- Roadmap.** The four IMRaD blocks on one line. Tell the panel that Q&A comes at the end, but that clarifying questions are welcome anytime.

I I — Introduction (20 min)

Goal: by slide 11 the panel should know **what problem we're solving, why existing tools don't solve it, and exactly what we promised (the 12 objectives).**

-
- 3 **The problem.** Repetitive GUI work (data entry, copying between apps, email triage) eats hours. Each person does it *their own way*.
-
- 4 **Why current tools fall short.**
- Traditional RPA is scripted by hand, breaks when the UI changes, and needs a developer.
 - Generic computer-use agents are not personalized. They do the task *their way*, not the user's.
-
- 5 **The gap.** No tool *watches a user*, learns *their* workflow, and repeats it.
-
- 6 **Our idea in one sentence.** Record demonstrations, then a Transformer learns *where* to act and an LLM supplies *what* to type. Key point: it's not hardcoded per task.
-
- 7 **Background: behavioral cloning (BC).** A plain explanation: learn from (state, action) pairs, the way an apprentice copies a mentor. Mention its known weakness (small errors pile up) to set up DAgger later.
-
- 8 **Related work.** RPA tools, computer-use / VLM agents, and BC for UI tasks. One line each on how Intern differs.
-
- 9 **Research objectives (1–12).** Group them by dimension: Perception (1), Representation (2, 4), Learning (3, 5), Adaptability (6), Scalability (7), Execution & Integration (8, 9, 10), Evaluation (11, 12). Show each target (e.g. $\geq 90\%$ action accuracy, $\geq 85\%$ end-to-end completion).
-
- 10 **Scope: the three study cases.** Why these three: together they cover *data entry*, *cross-app transfer* and *conditional judgment*.
- Scope #1: car-insurance data-entry form (Notepad intake file $\rightarrow$ form)
 - Scope #2: grade spreadsheet $\rightarrow$ web portal, tested on 8 portal versions that each change one thing (labels, order, extra fields, scales...)
 - Scope #3: the boss's task list in one Notepad file — cold emails, inbox triage (reply / forward / schedule / leave alone) and checks (did someone reply, any mail about a topic, am I free) — drafts only, never auto-send
-
- 11 **Significance & limitations up front.** Who benefits. State what's out of scope (Windows only, three bounded tasks, no auto-send email).
-

M — Methods (35 min)

Goal: the panel should understand **how the system is built and how we measure it**, well enough to judge the results.

System overview (10 min)

-
- 12 **Architecture diagram.** Source window and target window $\rightarrow$ Observation $\rightarrow$ agent loop $\rightarrow$ Transformer + LLM + TaskPlugin $\rightarrow$ ActionExecutor (pyautogui). Use the diagram from `DEVELOPERS.md` $\rightarrow$ `How It Works`.

-
- 13 The human-only rule.** The agent only sees the screen and uses the mouse/keyboard: no file shortcuts, no app scripting. Explain why this makes the claim stronger.
-
- 14 Division of labor.** The Transformer decides WHERE (which element, what order) and WHAT action (click / type). The LLM gives the value. The widget type decides HOW (a recorded design decision; explain why: click-vs-type is universal plumbing, not personalization). For Scope #3 the same idea, in words: **habits first** (a trained model of how the user handled similar emails, then their history with that sender), and the LLM only for what habits can't settle. Play offers Habits only / Habits + reasoning / Reasoning only, so the panel can see the difference.

Pipeline, step by step (15 min)

-
- 15 Step 1: Record.** The DemoRecorder takes UIA snapshots on every click and filters out junk clicks.
-
- 16 Step 2: Clean.** `clean_demos.py` drops dropdown-selection clicks, off-window noise and duplicates.
-
- 17 Step 2b: Oversample the finish.** Tail-oversampling teaches the model to click Submit, with no hardcoded completion rule.
-
- 18 Step 3: Train.** `TransformerAgentNetwork`: causal transformer, per-element features (bbox, role, `is_focused`, `is_filled`, 384-d text embedding), and pointer heads with LayerNorm.
-
- 19 Step 4: Run.** The observe → predict → act → re-observe loop runs until done or `max_steps`.
-
- 20 Correction loop (Dagger).** On a failure, the agent watches the user fix it and saves that as a new trace. Explain it plainly.

Key engineering decisions (5 min). Pick 3–4; the rest go to backup.

-
- 21 Perception is an adapter.** UIA and Excel speak one shared element schema, and the agent fails loudly on a mismatch.
-
- 22 The `is_filled` feature.** Fixed looping by letting the model see which fields are already done.
-
- 23 Action-space collapse to {click, type}.** Action-type accuracy went from 50% to 80%.
-
- 24 No form-specific hardcodes (`ScopeConfig`).** The agent doesn't know which application it's running in.

Evaluation design (5 min)

-
- 25 Metrics per objective.** A table listing objective → metric → script that computes it (`objective_metrics.py`, `bc_fidelity.py`, `validate_transitions.py`, `encoding_ambiguity.py`, `eval_metrics.py`).
-
- 26 Verdict rules.** MET / PARTIALLY MET / NOT MET / NOT EVALUATED / NOT APPLICABLE. An objective is MET only if *every* measured scope met it. NOT EVALUATED (no data) ≠ NOT MET (failed).

- 27 Clone test (generalization check).** Train on top-down order, then on bottom-up. If each reproduces its own order, it's cloning, not memorizing.

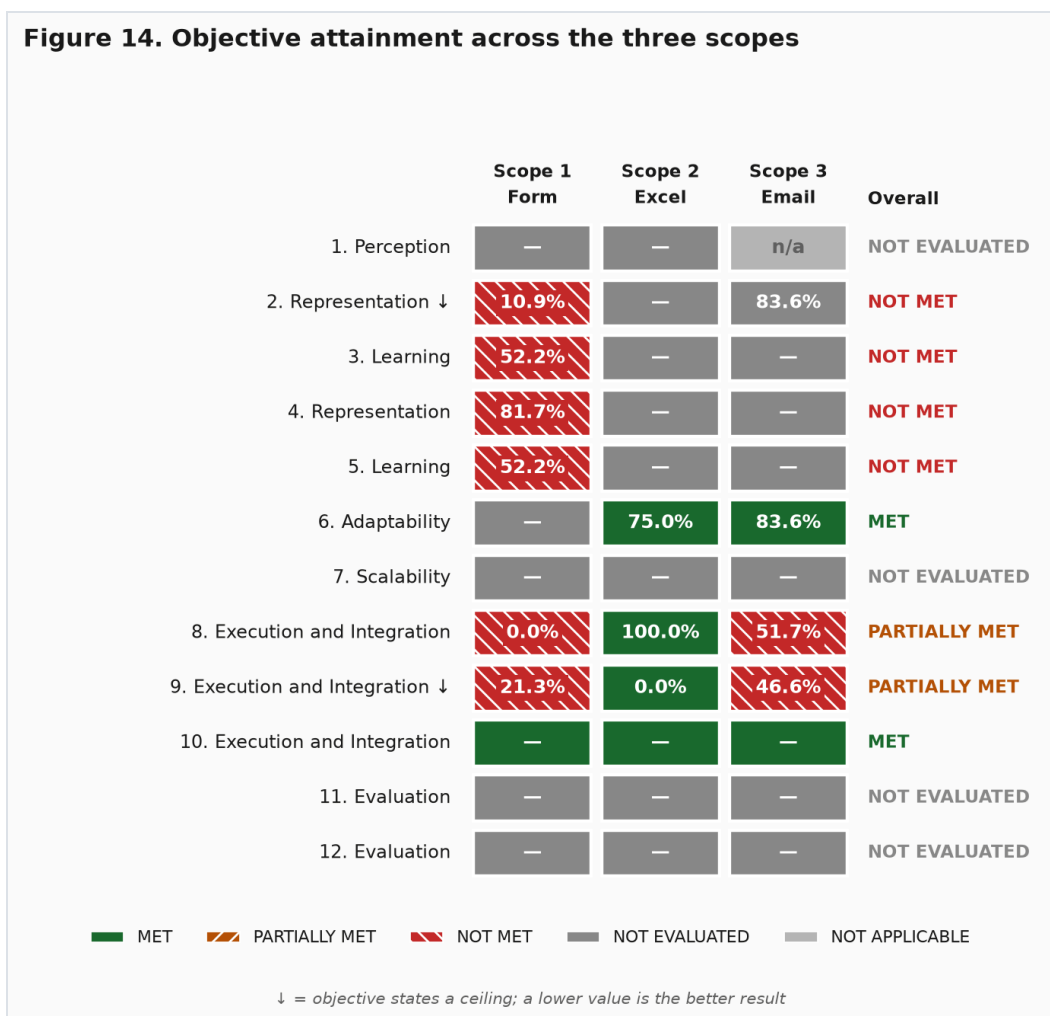
Break / buffer (5 min)

Use it, or use it to catch up if Methods ran long.

R — Results (27 min)

Goal: **report what the logs say**, objective by objective. Show the summary first, then the details. Use the figures in `scripts/thesis_figures/output/` (`fig_matrix.png`, `fig_obj01.png` ...).

- 28 Headline result first: the objective × scope matrix** (`fig_matrix.png`). The panel sees the whole picture before the details.



Slide 28: the objective × scope matrix, regenerated from the metric files on the date above.

- 29 Proof of cloning.** Top-down order → 74% exact match; bottom-up order → 93%. Same architecture learned two opposite orders, so it's genuinely cloning.

- 30 Demo (3–4 min).** Every scope works the same way in the app: **Launch** opens the real target (form, portal, inbox + task list), **Play** works in that same window.
- Scope #1: intake file → car-insurance form.
 - Scope #2: grade sheet → portal, every field and the pass/fail rule filled.
 - Scope #3: Launch resets the practice data and opens the inbox and `task_list.txt`; Play drafts the cold emails (signed Kevin), handles every inbox email, then answers the checks; new "Scheduled" lines appear at the end of the task list.
- Have a pre-recorded video ready as a backup.** Press Scope #3's Launch about a minute early (the first sort takes ~45 s), and make sure LM Studio is running with a model loaded.

Objective by objective (one slide each, ~1.5 min)

Slide	Objective	Target	Measured (2026-10-06)	Verdict
31	1 Vision perception	≥ 95% detection	—	NOT EVALUATED
32	2 Encoding ambiguity	< 5%	S1 10.9% (n=19)	NOT MET
33	3 & 5 Action prediction	≥ 90%	S1 52.2% (n=6,401)	NOT MET
34	4 State-transition mapping	≥ 90%	S1 81.7% (n=19)	NOT MET
35	6 Adaptability	≥ 75%	S2 75.0% (n=24), S3 83.6% (n=292)	MET (zero margin on S2)
36	7 Scalability	≥ 90% maintained	—	NOT EVALUATED
37	8 End-to-end completion	≥ 85%	S1 0.0% (n=197), S2 100% (n=210), S3 51.7% (n=292)	PARTIALLY MET
38	9 Execution error	≤ 10%	S1 21.3% (n=8), S2 0.0% (n=210), S3 46.6% (n=292)	PARTIALLY MET
39	10 Integration	qualitative	all three scopes	MET
40	11 & 12 vs RPA + significance	> 10–20%, p < .05	— (tool ready, no RPA runs yet)	NOT EVALUATED

- 41 Results summary.** Count of objectives by verdict. Say it plainly: "Here's what works, here's what doesn't, and here's what we didn't measure yet."

Before the defense: slides 31, 36 and 40 are currently empty. If you collect the RPA comparison data (`scripts/compare_baseline.py` is ready) or scalability runs before the defense, regenerate the table. If not, present them as NOT EVALUATED and explain why. Don't fill them with estimates.

D — Discussion & Conclusion (15 min)

Goal: **explain what the results mean**, own the weak spots, and close strong.

-
- 42 What the results mean.** Personalized cloning is real (slide 29). The pipeline works end to end on Scope #2 (100% completion). Scope #1 is held back by navigation/action accuracy, not by the architecture.
-
- 43 Why Scope #1 completion is 0%.** The real causes, in plain words: errors pile up over a long 176-field / 8-tab form, cold-start, and rare tab-switching events. Connect each one to a known BC weakness.
-
- 44 Why ambiguity / action accuracy miss target.** Small n on ambiguity (19), and the LLM handles value steps by design, which limits how high the Transformer's accuracy can go.
-
- 45 Adaptability, read honestly.** It's MET, but only at the threshold. n=24, and only 3 of 8 variants were fully solved.
-
- 46 Limitations.** Windows/UIA only, small evaluation sizes, no RPA baseline yet, Scope #3 on mock data, pixel-coordinate brittleness.
-
- 47 Recommendations / future work.** DAgger correction loop, automatic rare-event weighting (inverse frequency / focal loss), vision perception adapter, a larger recording campaign, the RPA comparison study.
-
- 48 Contribution.** A personalized, demonstration-learned GUI agent with a swappable perception layer. Scripted RPA doesn't learn, and generic agents don't personalize. This one does both.
-
- 49 Conclusion.** Three sentences: the problem, what we built, and what we showed. Then thank the panel.
-

Q&A (15 min)

50 "Questions?" slide. Keep the matrix (`fig_matrix.png`) visible behind it.

Backup slides (show only if asked):

- B1. Full per-element feature list and model hyperparameters
- B2. Option A (pure transformer) vs Option B (division of labor): why B
- B3. DAgger implementation details
- B4. Inbox Router internals: rules first, LLM only when rules can't decide, drafts only (never auto-send)
- B5. Each metric's exact formula and source script
- B6. Old Chapter 4 numbers vs current: why they changed (the old numbers came from the pre-fix system)

Likely panel questions: prepare a one-line answer for each.

- "Isn't this just RPA with extra steps?" → RPA is scripted; ours learns from a user.
- "Why does Scope #1 fail end-to-end if cloning works?" → slide 43.
- "Is 75% adaptability meaningful at n=24?" → slide 45: it's a weak pass, and we say so.
- "Where's the comparison to RPA?" → the tool is built; the data isn't collected yet.
- "How much is hardcoded?" → ScopeConfig + widget→action rules; navigation is 100% learned.
- "Why use an LLM at all?" → it reads the source data and gives the value; the Transformer gives the personal workflow.

- "What are 'habits' in Scope #3?" → what Intern learned from how *you* handled email: a trained model of similar past emails, then your history with that sender. Habits only = pure cloning; anything unsure is left for you.
- "Does it send email on its own?" → no. Everything is a draft or a calendar entry; there is no send function anywhere in the code.

Speaker split (if presenting as a group): assign one person per IMRaD block so handoffs happen at block boundaries, not mid-section.